computers

MDPI

Article

Evaluating LLMs for Source Code Generation and Summarization Using Machine Learning Classification and Ranking

Hussain Mahfoodh [1], Mustafa Hammad [2,*], Bassam A. Y. Alqaralleh [3] and Aymen I. Zreikat [4]

[1] Independent Researcher, Manama, Bahrain; hussain.mahfoodh@ieee.org
[2] Department of Software Engineering, Mutah University, Mutah 61710, Jordan
[3] College of Business Administration, American University of the Middle East, Egaila 54200, Kuwait; bassam.alqaralleh@aum.edu.kw
[4] College of Engineering and Technology, American University of the Middle East, Egaila 54200, Kuwait; aymen.zreikat@aum.edu.kw
* Correspondence: hammad@mutah.edu.jo

Abstract

The recent use of large language models (LLMs) in code generation and code summarization tasks has been widely adopted by the software engineering community. New LLMs are emerging regularly with improved functionalities, efficiency, and expanding data that allow models to learn more effectively. The lack of guidelines for selecting the right LLMs for coding tasks makes the selection a subjective choice by developers rather than a choice built on code complexity, code correctness, and linguistic similarity analysis. This research investigates the use of machine learning classification and ranking methods to select the best-suited open-source LLMs for code generation and code summarization tasks. This work conducts a comparison experiment on four open-source LLMs (Mistral, CodeLlama, Gemma 2, and Phi-3) and uses the MBPP coding question dataset to analyze code-generated outputs in terms of code complexity, maintainability, cyclomatic complexity, code structure, and LLM perplexity by collecting these as a set of features. An SVM classification problem is conducted on the highest correlated feature pairs, where the models are evaluated through performance metrics, including accuracy, area under the ROC curve (AUC), precision, recall, and F1 scores. The RankNet ranking methodology is used to evaluate code summarization model capabilities by measuring ROUGE and BERTScore accuracies between LLM code-generated summaries and the coding questions used from the dataset. The study results show a maximum accuracy of 49% for the code generation experiment, with the highest AUC score reaching 86% among the top four correlated feature pairs. The highest precision score reached is 90%, and the recall score reached up to 92%. Code summarization experiment results show Gemma 2 scored a 1.93 RankNet win probability score, and represented the highest ranking reached among other models. The phi3 model was the second-highest ranking with a 1.66 score. The research highlights the potential of machine learning to select LLMs based on coding metrics and paves the way for advancements in terms of accuracy, dataset diversity, and exploring other machine learning algorithms for other researchers.

Keywords: large language models; LLM applications; code generation; summarization; Halstead complexity; linguistic similarity

Check for updates

Academic Editor: Xiaorui Shao

Received: 20 December 2025
Revised: 29 January 2026
Accepted: 3 February 2026
Published: 10 February 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The use of large language models (LLMs) has increased within the software development community. LLMs are a type of natural language processing (NLP)–based artificial intelligence that understands and generates content from human language input command prompts, making them invaluable for tasks like translation, code generation, summarization, and sentiment analysis. Many software engineers rely on LLMs for software development tasks. Writing code, test cases, debugging, and understanding software code are some of the applications that software developers use with LLMs. This significantly enhances software engineers' productivity and assists in automating tasks that reduce the time spent on repetitive coding tasks and allow engineers to focus on more complex and creative aspects of development.

Code generation applications are essential for enhancing the efficiency and productivity of software development. They automate the creation of code, reducing the time developers spend on repetitive and boilerplate tasks. However, when using these applications, it is crucial to consider the complexity of the code being generated to ensure it aligns with the project's requirements and maintainability standards. Code correctness is another key factor, as the generated code must function as intended without introducing bugs or loopholes. Accuracy is equally important for ensuring that the code adheres to best practices and coding standards. Balancing these considerations helps developers leverage code generation tools effectively while maintaining high-quality and reliable software.

Code summarization applications are vital for improving the readability and maintainability of software projects. They help developers quickly understand the purpose and functionality of code segments, which is especially useful when dealing with large codebases, understanding legacy code, debugging, onboarding new team members, and improving documentation. However, ensuring the accuracy of these summaries is crucial, as any misinterpretation can lead to misunderstandings and potential errors in the code. Additionally, maintaining linguistic similarity between the summary and the original code is important to preserve the context and intent, making the summaries more intuitive and easier to follow. Balancing these considerations ensures that code summarization tools provide reliable and helpful insights for developers.

Evaluating LLMs for code generation and summarization is critical, as these models increasingly influence software engineering practices. Proper selection of LLMs can impact code quality, maintainability, and development efficiency. In industry settings, selecting an appropriate LLM can significantly improve code quality and reduce debugging time. For example, a software development team integrating an LLM for automated code suggestions observed that using a model with higher summarization and maintainability scores reduced error-prone refactoring and improved overall productivity. Such insights demonstrate the practical benefits of informed LLM selection for both small and large-scale software projects. By systematically assessing LLM capabilities, our study aims to provide actionable insights that help researchers and practitioners make informed decisions when integrating LLMs into software development workflows.

Selecting the right LLM for code generation and summarization is crucial due to the varying requirements of code complexity, correctness, differences in LLMs' perplexity [1], and linguistic dissimilarity. Closed-source LLMs often come with limitations such as a lack of transparency and customization, making it challenging to assess their suitability for specific tasks. Open-source LLMs, on the other hand, offer greater transparency and flexibility, allowing researchers and practitioners to inspect model architectures, training objectives, and fine-tuning strategies, which is particularly valuable for understanding model behavior in code generation and summarization tasks. This transparency enables systematic error analysis, reproducibility, and informed model selection. Additionally,

the flexibility of open-source LLMs allows customization through fine-tuning, prompt engineering, and integration into existing development pipelines, making them well-suited for diverse software engineering applications where task-specific adaptation is often required. However, the decision is complex and multifaceted, as it involves evaluating multiple parameters to ensure the chosen model can handle the intricacies of the code while maintaining high standards of accuracy and readability. This process requires careful consideration and balancing of these factors to make an informed and effective selection.

Despite all of the different available LLM applications, assessing both LLM code generation and code summarization is a complex process to measure, as it involves different aspects to evaluate the code quality generated and the code similarity levels reached. This study aims to leverage machine learning classification and ranking algorithms to conduct an empirical study to address this literature gap by proposing a framework for evaluating LLM code and summarization similarity using different quality measures. This research will analyze code generation and code summarization output from four open-source LLMs. Coding problems are given to these models to solve and analyze the code generated in terms of test case correctness, the quality of the generated code, the complexity of the software program generated, perplexity to assess LLM tokens to study the model's abilities, and code summarization text-similarity evaluation to verify if the models are suitable for summarizing a code problem and whether they can fulfill the project stakeholders' needs. LLM selection is implemented through classification accuracy and ranking score to demonstrate how to properly select the best-suited LLM for a specific coding task, considering features related to the generated code and code summary obtained from the LLM.

The major contributions of this study are as follows:

- A machine learning-driven framework combining classification (SVM) and ranking (RankNet) methods to evaluate open-source LLMs for code generation and summarization tasks.
- The systematic extraction and analysis of software engineering metrics and summarization similarity features to inform LLM selection.
- A comprehensive evaluation of four open-source LLMs using Python-based MBPP tasks, including comparative analysis with baseline methods.
- A discussion of practical implications for software engineering applications and guidance for future research on LLM selection.

The rest of this paper is organized as follows. Section 2 provides related work. Section 3 describes the methodology. Sections 4 and 5 present the experimental results and discussion. Finally, Section 6 presents the study's conclusion and limitations.

2. Related Work

Since their early emergence, LLMs have been used widely to assist in various software engineering tasks, including code generation. The study by Jiang, J. et al. [2] categorized code LLMs-related software applications into understanding and generation tasks. Examples of generation tasks consist of code generation and code summarization. Examples of understanding tasks include code classification, code search, and bug detection applications. The study discussed how the availability of LLMs has made software development more productive and accessible by extending automation tasks, which has led to more developers being dependent on different software-level code generation models despite their complexity levels [3]. The study by Haque A. [4] compared software project status before and after the emergence of LLMs. The study showed that projects relied heavily on manual processes, making them time-consuming, prone to errors, and resource-intensive. After the rise of LLMs, the outlook changed drastically, shifting projects towards automation, therefore

improving the speed of development, increasing code quality, and making them less prone to errors. The same study reported a survey of different AI-assisted software development tasks used by developers, from planning to maintenance tasks. The highest percentage was coding with 82.5% and debugging with 49%. The same study stressed the difference between closed- and open-source LLMs, indicating how closed models sometimes provide inefficient code where test quality is more related to code complexity. Another study by Das K. et al. [5] revealed that ChatGPT-generated code was used as-is by developers and resolved only 5.83% of the total issues. Moreover, LLMs may also produce different results for the same code generation prompt, as shown in studies [6–8], which makes the use of LLMs in code generation tasks unpredictable in terms of correctness.

The quality of LLM-generated code is vital for any software project to obtain a high-quality code base. The work by Sepidband M. et al. [9] studied code complexity metrics for LLMs' code generation output for four datasets, and compared the pass@1 score for four LLMs. The study analyzed the distributions of complexity metrics and how they differ between successful and failed code solutions generated by LLMs. Another study by Khan M. et al. [10] analyzed how complexity metrics differ between ChatGPT-generated code and human-written code. The machine learning models used were able to distinguish ChatGPT from human code with up to 88% accuracy. The study by Hu W. et al. [11] proposed a complexity-aware metric benchmark to evaluate LLM code generation. The study uses nested call graphs to categorize the difficulty of code problems and generate benchmarks to be compared with the ground truth. In the context of automated software synthesis, the study by Biswas S. et al. [12] reported that LLMs often face the challenge of local optima, where the model generates functional but suboptimal code or fails to navigate complex logical constraints, indicating the need for better LLM reliability selection criteria. Another study by Mahmud T. et al. [13] uses ensemble learning on LLM code generation tasks, using the CodeBLEU metric for detecting semantic similarity and CrossHair's differential behavior analysis to select the most reliable LLM solutions that achieve the highest accuracy of 90.2%. Other research by Huynh N. and Lin B. [14] discussed the need for CodeBLEU metrics to assist in evaluating the quality of LLM code generation, both grammatically and logically. The results of the study demonstrate that CodeBLEU has a better correlation with human evaluation scores compared to traditional metrics such as BLEU.

By analyzing the perplexity of generated code, insights into the complexity and predictability of the generated code can be determined. The study by Mostafa A. et al. [15] demonstrated the impact of tokenizers on model performance by using the Llama 3.2 (1B-parameter) model, the encoder-only BERT model, and the encoder–decoder BART-Base model on binary code using three different tokenizers. The highest Llama accuracy reached is 85.76%, and BERT's highest accuracy is 86.58%. The study by Xu J. [1] discussed how different models yield different perplexity scores based on how models focus on core logic, usage, boundary conditions, and robustness. Other factors include code authenticity, language diversity, and generalization capabilities. Another study by Xu Z. and Sheng V. [16] employed the GPT-3.5 model to detect AI-generated code from human-like code. The research showed that AI-generated code usually has a low perplexity score compared with human code, and no other LLMs were used in the study.

Several studies showed how beneficial it is to use LLMs with bug reports, making use of LLM text generation and text summarization features to provide well-structured bug reports. This will not only help to minimize ambiguity but also enhance developers' productivity by helping them resolve issues with efficient processes and clear guidelines [17]. The study by Bo L. et al. [18] utilized the ChatGPT GPT-3.5-turbo LLM to rewrite bug reports with missing information to produce complete and accurate bug reports, with the

goal of saving developers' time and allowing them to focus solely on software fix tasks. Some other studies [19] try to mitigate the issue of unstructured bug reports by creating structured bug reports with the use of LLMs' text generation prompts. The same study uses SBERT, ROUGE-1, and CTQRS score measures, with results reaching up to a 70% CTQRS score on unseen projects, with an overall AI approach that is able to generalize well on bug report datasets. Generating correct bug titles in bug reports is important for describing bug severity and bug urgency for fixing. The study by Surekaby A. et al. [20] discussed the linguistic importance of bug-report titles in assigning the correct bug severity level and ensuring correct interpretation by internal stakeholders. The same study used a predictive classifier to perform statistical analysis on word frequency and distributions across various severity levels. The use of LLMs to generate bug descriptions is mentioned in the work by Kang S. et al. [21], which found that a fifth of LLM-produced code comments contain inaccurate statements. The authors proposed a documentation testing-driven approach to test the correctness of LLM-generated code by comparing it with the accuracy of code comments. Similarly, LLMs are used as evaluators for bug reports. In the study by Kumar A. et al. [22], GPT-4o, LLaMA-3, and Gemini models were applied to bug title and bug description fields, and the results were compared with those of human evaluators. The study found that GPT-4o outperformed the other models for complex evaluations when compared with human evaluators, whereas human evaluators were more suitable for simpler evaluation tasks.

LLMs have been used widely with issue-tracking systems in providing bug descriptions, assigning labels [23], and in task prioritization [24]. The work by Chen Z. et al. [25] used ChatGPT and DeepSeek models on bug reports from the Apache Jira dataset. The authors indicated that LLMs show potential in assisting developers in decomposing complex bug reports, but their accuracy levels performed poorly. New LLM tools have also been used to access issue-tracking systems in the work by Ciepielowski L. [26], where the title, description, and comments for each bug issue are used as embedding vectors for the tool to retrieve bug-issue-related information. The study revealed that practitioners preferred using the chatbot tool to retrieve knowledge, which can also be used to retrieve bug details if bug titles are vague, and ultimately save developers time in overall searching efforts. In addition, ambiguity in bug reports has led stakeholders to use LLMs to clarify bug reports, similar to the study done by [27] to recreate the test cases from bug reports, where only 50% of the executable test cases were obtained across all the study projects.

LLM code summarization is also essential to developers when there is a need to understand complex code or identify certain bugs within the LOC. The work by Sun W. [28] used summary-to-summary and summary-to-code similarities generated from four LLMs, compared with the reference text summary and the code snippet reference summary. The study results show that CodeLlama outperformed advanced GPT-4, StarChat, and GPT-3.5 combined in generating summaries. The study used different similarity metrics such as BLEU, METEOR, ROUGE-L, and BERTScore. Furthermore, LLMs can also be used as code summarization evaluators, as shown in the study by Wu Y. et al. [29]. The study used different LLMs from the GPT-3.5 and GPT-4 series to create a novel code summarization evaluation metric, which achieved 81.59% Spearman correlation with human evaluations, outperforming the existing BERTScore metric by 17.27%. The study indicated that the two models used varied in results, especially in accuracy and performance, and suggested using other LLMs as evaluators whenever possible.

A variety of studies explained the criteria to select the best LLM for different problems that are given. The work by Mbaiossoum B. and Bamana A. [30] provided a state-of-the-art overview combined with a comparative analysis of well-known LLMs. These guidelines are based on model size, performance, required resources, and ethical considerations. Another

study by Cheng J. [31] considered selecting the optimal LLM for code generation problems, only focusing on the cost perspective and task difficulty. The approach achieved a 7.86% improvement in pass@1 score while reducing resource consumption by 88.9% compared to the baseline used. Within code generation problems, other applications of LLMs include selecting the best hyperparameters [32], the best programs [33] from different LLM outputs through consistency and LLM-generated test suite criteria, and even performing code validation from code features [34].

Selecting the best open-source LLM for code generation and code summarization involves evaluating several key factors. Considering accuracy, clarity, and error detection is essential for evaluating the rationale behind the correct selection. In this research, we will analyze how to make judgments about LLM selection that best meet the needs for code generation and code summarization by running specific code test tasks and evaluating complexity, correctness, accuracy, and linguistic similarity as features for LLM classification and ranking problems.

3. Methodology

In the following subsections, a discussion of the test case dataset used in this experiment, with its data type, structure, and volume, is provided; then, this study demonstrates the overall experiment workflow approach used. Furthermore, this study discusses the code quality metrics used to evaluate the generated code from each selected LLM, along with the text summarization similarity measures used. Moreover, the feature selection criteria used for machine learning classification and ranking problems are explained. Finally, experimental results and a comparison of LLMs are illustrated using the selection criteria.

3.1. Data Collection

This research uses the MBPP (Mostly Basic Python Problems) dataset [35], which contains entry-level programming tasks along with their unit test cases. This study uses a sanitized version [36] of the dataset, which is a cleaned and curated version of the original MBPP dataset. Table 1 lists the dataset attributes definitions, including their data types. The MBPP dataset is a benchmark designed to evaluate Python programming skills and code generation models. Each problem in this dataset includes a task description, a code solution, and a test case to verify correctness. This dataset is a valuable resource for benchmarking code generation models and improving their performance in solving real-world programming tasks.

Table 1. The MBPP dataset structure used.

Attributes	Definition	Type
source_file	Contains the full Python source code for the task.	String
task_id	A unique identifier for each programming problem.	Positive Integer
prompt	A natural language description of the programming task.	String
code	The reference solution code written in Python.	String
test_imports	Lists any Python modules that need to be imported to run the test cases or the solution code.	String
test_list	A list of test cases used to validate the correctness of the generated or reference code in the form of input–output pairs or assert statements.	String

The study incorporated a total of 427 programming tasks, consisting of 1324 test cases distributed among all of the tasks.

3.2. Overall Workflow Approach

Figures 1–3 illustrate the streamlined workflow of our study. Figure 1 shows the code generation approach used. Initially, this research collects coding problems and asks a specific LLM to solve the assigned problem. The dataset test cases are applied to the generated code, where different evaluation metrics are used on the generated code snippet to calculate correctness, code quality, complexity, and LLM perplexity.

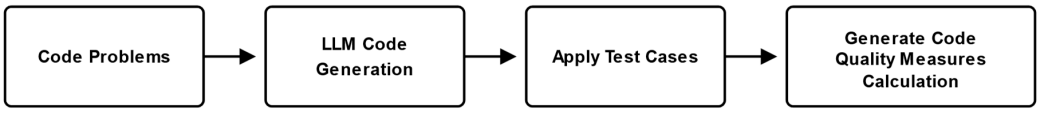

Figure 1. Code generation experiment workflow.

Figure 2 explains the code summarization methodological approach used. First, generated code snippets are collected from the earlier LLM output given. Second, an LLM is asked to explain and summarize the code given through text. Third, different linguistic similarity metrics are applied to compare the generated summary text with the code problem question text as an ideal reference text.

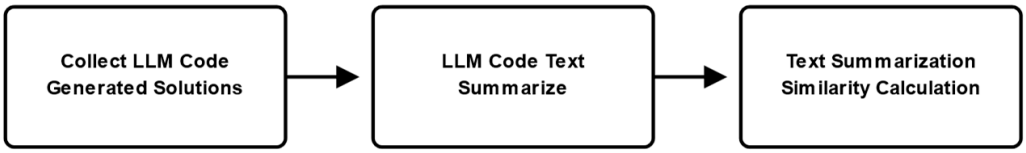

Figure 2. Code summarization experiment workflow.

Figure 3 demonstrates the classification problem used. From Figures 1 and 2, some of the features are collected from the evaluation results obtained from the code generation and code summarization experiments. Then, a classification problem is conducted on individual features in one case, and in another case, a ranking feature comparison is implemented. Finally, the performance of the classification and ranking methods is evaluated, on which the LLM selection criteria will be based. This study's implementation and dataset are publicly available on GitHub [37].

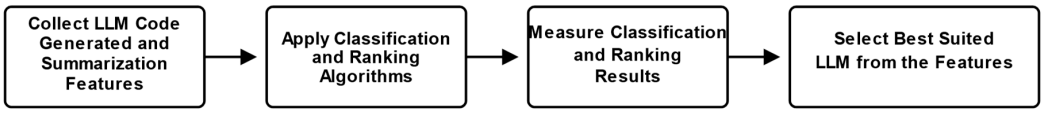

Figure 3. Classification and Ranking problem experiment workflow.

3.3. LLM Models

Four LLMs are selected and evaluated. Each model is used in the two code generation and code summarization experiments. The LLMs are selected based on a set of practical and methodological criteria. First, all models are open-source and publicly available, ensuring transparency and reproducibility. Second, they represent diversity in architectural design, parameter scale, and training objectives, including general-purpose language modeling and code-oriented pretraining. Third, these models are commonly adopted by the software engineering community for code generation and reasoning tasks, making them representative of realistic deployment scenarios. Larger proprietary or highly specialized models are excluded to avoid reproducibility constraints and unfair computational advantages. In addition, their overall size is compatible with the experiment's limited machine storage resources. Collectively, these four models provide a balanced and sufficient sample for evaluating the effectiveness of machine learning-based classification and ranking techniques

for LLM selection. Table 2 presents a summary of the LLMs used in this study, including their model size and context length.

Table 2. Summary of the LLMs used in the study.

Model Name	Provider	Model Size	Context Length
Mistral [38]	Mistral AI	7 billion parameters	8K tokens
Gemma 2 [39]	Google	2 billion parameters	8K tokens
Phi-3-mini [40]	Microsoft	3.8 billion parameters	4K and 128K tokens
CodeLlama [41]	Meta	7 billion parameters	16K tokens

Mistral is a family of high-performance language models developed by Mistral AI, offering scalable and efficient models like Mistral Medium and Codestral for general reasoning and code generation. Gemma 2, created by Google, is a lightweight open-source model optimized for text generation and reasoning, designed to run efficiently on modest hardware with advanced context handling. Phi-3-mini, from Microsoft, is a compact 3.8B-parameter model focused on reasoning, math, and code, trained on synthetic and filtered data, and capable of handling long contexts with instruction tuning. CodeLlama, developed by Meta, is a code-specialized model based on the Llama 2 architecture, supporting large contexts and multiple programming languages, ideal for tasks like code generation, completion, and debugging.

3.4. Code Generation Approach

Figure 4 shows the process of the code generation experiment. The process starts with providing the dataset JSON as a source file and retrieving the coding question. As shown in Figure 4, the method name is retrieved from the reference code solution obtained from the dataset. This method name is used as part of the prompt question in order to have a similar code output generated to the reference solution obtained, and to avoid large deviations in the code quality evaluation results. By selecting a specific LLM, a prompt question is given to the LLM to retrieve the coding solution. Then, the code solution is assessed using the retrieved test cases from the same dataset task.

The number of passed, failed, and error test cases is stored for each task. If the code obtained from an LLM cannot be executed, it is discarded and is not considered in the overall result calculation. A failed code execution might mean that the LLM misunderstood the task, that unwanted text was provided by the LLM, or that the code generated has a syntax error that makes the code non-executable. Not to be confused with a failed test case, a failed test case is considered in the evaluation calculation since it means the code is executable, but it does not cover all the test cases provided.

The error-free code solution is further evaluated using code quality measures based on syntax, semantics, lexical, and logical properties.

Code snippet Listing 1 shows an example of the prompt question used, where the generated code is obtained for each given coding task. The prompt question asks the LLM to only output the solution code. The output code is examined again and cleaned by removing any unwanted descriptions, backticks, indentation, and grave accents that the LLM might produce along with the code, which could affect the code evaluation calculation.

All experiments were conducted under controlled context-length constraints to ensure fair comparison across models. The MBPP dataset samples fall within the supported context windows of the evaluated LLMs, and prompt truncation was not required. In scenarios involving longer contexts, commonly used strategies, including truncation, sliding-window segmentation, and chunk-based prompting, were not used in this experiment.

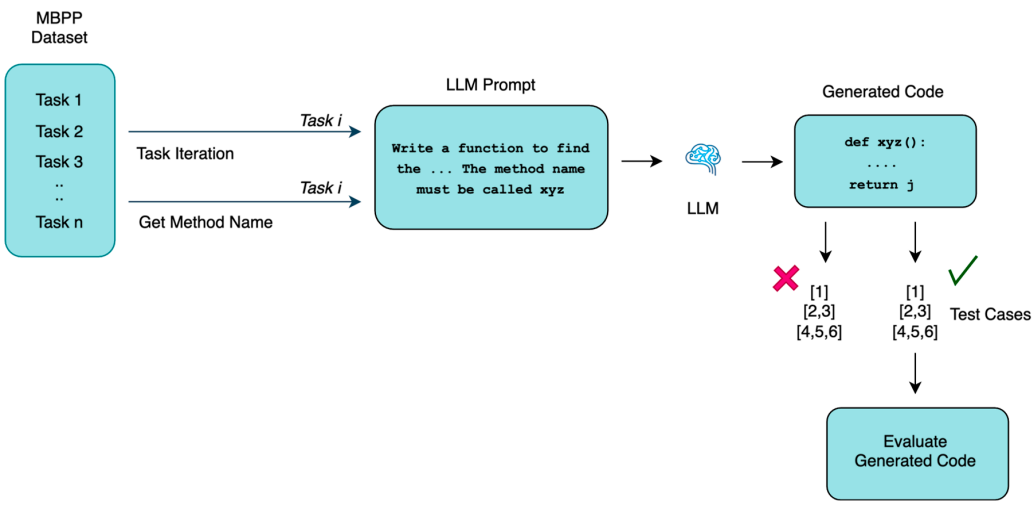

Figure 4. LLM code generation process.

Listing 1: LLM, Code Generation Template Prompt Question used in Python Programming Language.

```
promptQuestion = "Write a function to find the shared elements from the given two
    lists."
 question = f"""{promptQuestion} The method name must be called
     {extract_method_name_from_json(code_from_json)} and show only code and do not
     enter any other text or indentation or any backtick or grave accent. Make the
     code at least four lines. """
```

3.5. Code Generation Evaluation Metrics

The task code generated from the LLM is obtained and evaluated. Halstead complexity, Maintainability Index, raw metrics, and cyclomatic complexity are used to evaluate the code structure, complexity, and maintainability of the generated code. Table 3 shows an overview of the Halstead metrics used [42] as features to be evaluated for the LLM code generated, including their purpose and formulas.

In order to calculate the quality of the generated code against the solution reference code given in the dataset, a simplified version of the CodeBLEU metric is used, which is tailored for evaluating lexical code similarity. The method splits the code into tokens and measures the overlap of token sequences using N-gram precision, where the method uses equal weights for each n-gram level. Furthermore, the method uses the geometric mean to combine precision scores stably, applying a brevity penalty in case of a short candidate code compared to the reference. Formulas (1)–(3) show N-gram precision, geometric mean, and brevity penalty formulas, respectively. Formula (4) shows the final BLEU-like score retrieved between the generated code and the solution reference code.

$$P_n = \frac{\sum_{\text{ngram} \in C_n} \min(\text{count}_{C_n}(\text{ngram}), \text{count}_{R_n}(\text{ngram}))}{\sum_{\text{ngram} \in C_n} \text{count}_{C_n}(\text{ngram})} \tag{1}$$

where

- C_n = candidate n-grams of length n
- R_n = reference n-grams of length n

$$GM = exp(\sum_{n=1}^{N} w_n . ln(P_n)) \tag{2}$$

where

- w_n = weight for n-gram level n (default: $1/N$)
- P_n = precision for n-gram level n

  If any P_n= 0, then GM = 0

$$BP = \begin{cases} 1, & \text{if } c \geq r \\ e^{1-\frac{r}{c}}, & \text{if } c < r \\ 0, & \text{if } c = 0 \end{cases} \tag{3}$$

where

- c = number of tokens in the candidate code
- r = number of tokens in the reference code

$$Score = BP * GP \tag{4}$$

A simplified version of CodeBLEU was employed in this study to reduce computational overhead and dependency complexity while maintaining consistency across models. Since the objective of the summarization experiment is comparative ranking rather than absolute functional correctness, the simplified metric provides sufficient discriminatory power.

Table 3. Halstead complexity metrics used in the experiment.

Metric	Description	Formula
Halstead Report	Code complexity & effort metric.	• η_1 = the number of distinct operators • η_2 = the number of distinct operands • N_1 = the total number of operators • N_2 = the total number of operands • Program Vocabulary: $\eta = \eta_1 + \eta_2$ • Program Length: $N = N_1 + N_2$ • Calculated Estimated Program Length: $\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$ • Size of the Implementation (Volume): $V = N \times \log_2 \eta$ • Difficulty to Understand or Write Code (Difficulty): $D = \frac{\eta_1}{2} \times \frac{N_2}{\eta_2}$ • Estimated Mental Effort to Develop the Code (Effort): $E = D \times V$ • Number of Bugs Expected in the Program (Bugs): $B = \frac{E^{2/3}}{3000}$ • Estimated Time Taken to Write the Code (Time): $T = \frac{E}{18}$
Maintainability Index	A composite metric that estimates how maintainable a piece of code is.	• The score for the Maintainability Index ranges from 0 to 100. • The formula combines the following three software metrics: – V: Halstead volume – CC: Cyclomatic complexity – LOC: Lines of code The Maintainability Index (MI) is calculated using the following simplified formula: $\text{MI} = \max\left(0, (171 - 5.2 \cdot \ln(V) - 0.23 \cdot CC - 16.2 \cdot \ln(\text{LOC})) \cdot \frac{100}{171}\right)$
Raw Metrics	Basic code statistics that give a quick overview of the code structure.	• LOC: Lines of code • LLOC: Logical lines of code (ignores comments, blank lines) • SLOC: Source lines of code • C: Number of comment lines • M: Multi-line comments • B: Blank lines • S: Single-line comments
Cyclomatic Complexity	Measures the number of independent paths through the code.	• Cyclomatic Complexity (CC): Calculated based on the control flow graph. $\text{CC} = E - N + 2P$ where – E = Number of edges in the control flow graph. – N = Number of nodes in the control flow graph. – P = Number of connected components (typically $P = 1$ for a single program or function).

Besides calculating passed, failed, and error test cases, the *Pass@1* metric is used to evaluate the accuracy of the generated code and its correctness. Formulas (5) and (6) show the *Pass@1* and *Pass@k* calculations.

$$Pass@1 = \frac{\text{Count of problem for passed first solution}}{\text{Total number of problems}} \tag{5}$$

$$Pass@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \tag{6}$$

where

- n = total number of generated samples per problem
- c = number of correct samples (i.e, samples that pass all tests)
- k = number of samples evaluated (e.g., top-k)
- $\binom{a}{b}$ = binomial coefficient ("a choose b")

To evaluate the LLM's prediction confidence, the perplexity measure is used on the generated code. Perplexity is a measure of how well a language model predicts a sample. Lower perplexity means that the model is more confident in its predictions.

Given a sequence of tokens $x_1, x_2, \ldots, x_N$, the mathematical definition of perplexity is shown in Formula (7), where $P(x_i \mid x_{<i})$ is the probability of token x_i given the previous tokens.

$$\text{Perplexity}(x) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(x_i \mid x_1, x_2, \ldots, x_{i-1})\right) \tag{7}$$

Formula (8) shows the loss returned by a given language model, which is typically the average negative log-likelihood (NLL).

$$\text{Perplexity} = \exp(NLL) \tag{8}$$

The code generated for each task from each model in Table 2 is tokenized and evaluated using three other LLMs. Table 4 shows a summary of the LLM perplexity evaluators used in the study. The evaluator models used are of the TensorFlow-native type [43].

Table 4. LLM perplexity evaluators used in the study.

Model Name	Provider	Model Size	Context Length
microsoft/CodeGPT-small-py [44]	Microsoft	~110 M params	1024 tokens
gpt2 [45]	OpenAI	124 M params	1024 tokens
Distilgpt2 [46]	Hugging Face	82 M params	1024 tokens

CodeGPT-small-py is a lightweight GPT-Neo-based model developed by Microsoft, specifically fine-tuned for Python code generation and completion. It was trained on a large, cleaned dataset of Python files from GitHub and is designed to be efficient and comparable in performance to models like Codex for similar model sizes. GPT-2, created by OpenAI, is a general-purpose language model trained on a massive corpus of English text using a causal language modeling objective. It predicts the next word in a sequence and is widely used for text generation, with 124 M parameters in its smallest version. DistilGPT2 is a compressed version of GPT-2 developed by Hugging Face using knowledge distillation. It has 82 M parameters and retains much of GPT-2's performance while being faster and more resource-efficient, making it suitable for lightweight applications in text generation.

The study used fixed variables of *WINDOW_SIZE* with a default value of 512 and *STEP_SIZE* with a default value of 256. *WINDOW_SIZE* is the maximum number of tokens the model can handle at once, whereas *STEP_SIZE* is how many tokens to slide the window by for each new chunk. If the code is shorter than the *WINDOW_SIZE*, the study calculates perplexity for the whole code. The study averages the perplexity values from these three models and considers the result a single feature.

3.6. Code Summarization Approach

Figure 5 shows the code summarization process. The task code generated from the LLM is obtained, and for each task, one LLM is selected and asked to summarize a code snippet to be compared with the prompt question string given in the dataset. As shown in Figure 5, the summarization generated from the LLM will be used as-is for the comparison with the reference coding question obtained from the dataset and will be evaluated using textual summarization similarity metrics.

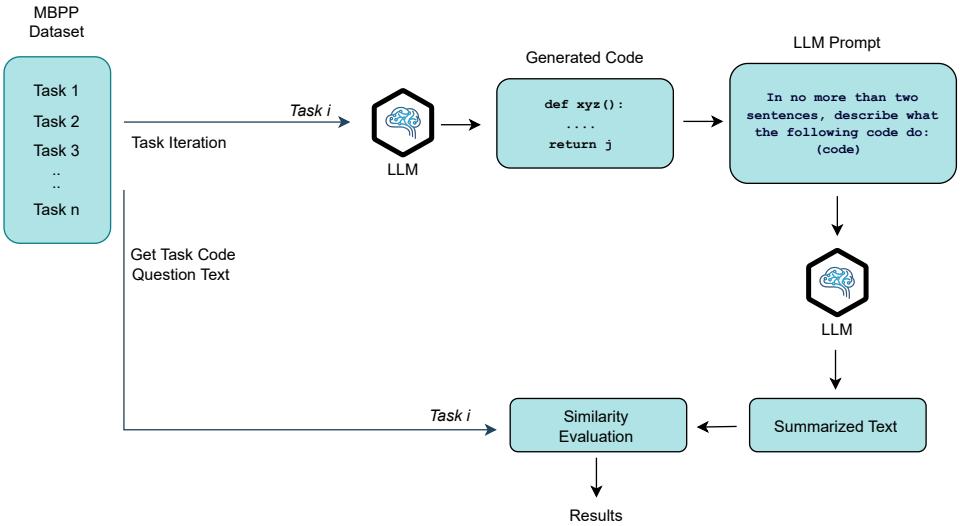

Figure 5. LLM code summarization process.

Text similarity methodology involves comparing a candidate text (e.g., a generated summary) to a reference text (e.g., the ground truth dataset question) to evaluate how close they are in meaning, structure, or wording.

Code snippet Listing 2 shows an example of the prompt question used to ask an LLM for the code summarization for a given task code.

Listing 2: LLM Code Summarization Template Prompt Question used in Python Programming Language.

```
question = (
    "In no more than two sentences, describe what the following code does:"
    f"{generated_code}"
)
```

3.7. Code Summarization Evaluation Metrics

Consideration of word-level, phrase-level, and semantic similarity, sentence structure, and fluency is needed for proper summarization evaluation. ROUGE metrics were selected to measure lexical overlap between generated summaries and reference descriptions, providing a baseline assessment of content coverage. BERTScore was included to capture semantic similarity using contextual embeddings, enabling evaluation beyond surface-level

matching. While ROUGE is sensitive to exact phrasing, it may underestimate semantically equivalent summaries. Conversely, BERTScore better captures semantic alignment but may overlook factual inaccuracies. Combining both metrics provides a more balanced evaluation of code summarization quality.

This study calculates similarity between the code text summary generated by an LLM and the code problem question reference from the dataset using ROUGE-1, ROUGE-2, ROUGE-L, and BERT Score metrics. Table 5 shows the rationale for selecting these metrics. The table lists the focus for each metric and its corresponding evaluation type. Formulas (9)–(20) show their corresponding mathematical definitions.

Table 5. Summarization evaluation metrics used in the study.

Metric	Type	Focus
ROUGE-1	N-gram recall	Measures unigram (word-level) recall—good for checking if key words are preserved.
ROUGE-2	N-gram recall	Measures bigram (phrase-level) recall—captures fluency and coherence.
ROUGE-L	Sequence-based	Measures longest common subsequence—reflects sentence structure and fluency.
BERTScore	Embedding-based	Uses contextual embeddings—captures semantic similarity, even with different wording.

Given R is the set of unigrams in the reference text and C is the set of unigrams in the candidate text, Formulas (9)–(11) show the ROUGE-1 mathematical representation for recall, precision, and F1 score, respectively.

$$\text{ROUGE-1 Recall} = \frac{|R \cap C|}{|R|} \tag{9}$$

$$\text{ROUGE-1 Precision} = \frac{|R \cap C|}{|C|} \tag{10}$$

$$\text{ROUGE-1 F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{11}$$

Given that R_2 is the set of bigrams in the reference text and C_2 is the set of bigrams in the candidate text, Formulas (12)–(14) show the ROUGE-2 mathematical representation for recall, precision, and F1 score, respectively.

$$\text{ROUGE-2 Recall} = \frac{|R_2 \cap C_2|}{|R_2|} \tag{12}$$

$$\text{ROUGE-2 Precision} = \frac{|R_2 \cap C_2|}{|C_2|} \tag{13}$$

$$\text{ROUGE-2 F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{14}$$

ROUGE-L is based on the Longest Common Subsequence (LCS) between the candidate and reference. Let $\text{LCS}(R, C)$ be the length of the longest common subsequence, $|R|$ be the length of the reference, and $|C|$ be the length of the candidate. Formulas (15)–(17) show the ROUGE-L mathematical representation for recall, precision, and F1 score, respectively.

$$\text{ROUGE-L Recall} = \frac{\text{LCS}(R, C)}{|R|} \tag{15}$$

$$\text{ROUGE-L Precision} = \frac{\text{LCS}(R, C)}{|C|} \tag{16}$$

$$\text{ROUGE-L F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{17}$$

BERTScore uses contextual embeddings from BERT to compare semantic similarity between tokens, capturing semantic similarity even when the wording differs.

Let $x_1, x_2, \ldots, x_n$ be the tokens in the candidate text and $y_1, y_2, \ldots, y_m$ be the tokens in the reference text. Let $\phi(x_i)$ and $\phi(y_j)$ be the BERT embeddings of tokens x_i and y_j, respectively. Formulas (18)–(20) show the BERTScore mathematical representation for recall (R), precision (P), and ($F1$) score, respectively.

$$R = \frac{1}{m} \sum_{j=1}^{m} \max_i \cos(\phi(y_j), \phi(x_i)) \tag{18}$$

$$P = \frac{1}{n} \sum_{i=1}^{n} \max_j \cos(\phi(x_i), \phi(y_j)) \tag{19}$$

$$F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{20}$$

This evaluation will only take into consideration the F1 score, as it is sufficient to evaluate the summarization text similarities since it balances both precision and recall values.

3.8. Model Classification Selection and Implementation

This section includes detailed implementation details of the LLM classification and ranking methodology used in our study. The following subsection's goal is to provide clarity and transparency on the approach used.

3.9. Feature Engineering

We extract 24 features from the code generation and code summarization experiments. All of these features are numerical. The features for the code generation experiment are taken from Table 3, with perplexity and CodeBLEU values added. The features consist of different scaling values, so the features are adjusted using min-max normalization to a fixed range from 0 to 1. The min-max scaling formula is shown in Formula (21).

$$\text{Scaled } F_n = \frac{F_n - \min(F_n)}{\max(F_n) - \min(F_n)} \tag{21}$$

The features for the code summarization experiment are the ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore F1 similarity scores calculated for each model, respectively.

3.10. Feature Selection

The selection of the features considers only the code-generation tasks that give a *Pass@1* score of 1 across all selected models, for the same tasks processed by each model; this indicates that only LLM outputs that passed all coding-task test cases without any failures or error occurrences are considered. Other features from the task results with any failed or error test cases in any model are discarded during feature selection among all models to avoid model bias in the final LLM selection results.

3.11. Model Selection and Configuration

Support vector machine (SVM) classification was selected for the code generation experiment. The rationale for the selection is due to its effectiveness in classification tasks and in handling high-dimensional feature spaces and limited sample sizes, which align with the characteristics of the extracted software metrics. SVMs are also well-suited for maximizing class separation when feature distributions overlap.

The SVM methodology used an approach to transform a one-dimensional, continuous dataset into a multi-class classification problem, where the primary focus was on data preparation and exploratory correlation analysis for a machine learning task, specifically a multi-class classification problem involving four different models (M1, M2, M3, M4). The logic first aggregates the individual feature vectors into combined feature matrices and creates corresponding class labels. Crucially, recognizing an inherent class imbalance in the dataset, it then employs the synthetic minority over-sampling technique (SMOTE) to artificially balance the class distribution. Finally, it calculates and visualizes the Pearson correlation matrix for the resampled features to identify and report the top 10 most strongly correlated, and thus potentially redundant, feature pairs.

A critical configuration setting in the SVM process is the probability configuration. This parameter is enabled to allow the model to not only predict the class but also output a probability estimate for each class. This is an essential step, as these probabilities are later used to calculate metrics like the area under the curve (AUC) and to generate the ROC plots, providing deeper insight into the model's performance beyond simple accuracy.

As the models used in the experiment could give different solutions with errors, and as this experiment considered only passed test cases, it is expected to have a different count of sample solutions provided by each model. In order to obtain accurate results for the classification problem, SMOTE was used to create synthetic samples and therefore create equal sample counts to reduce classification bias in the results.

While non-SMOTE approaches could provide better precision and calibration, given the data samples retrieved from LLMs in Table 6, which show class imbalance in the dataset, the model may not have enough data to learn the underlying pattern of the minority class, leading to a high probability of bias in class predictions.

Table 6. Results of code generation solutions provided by each LLM model.

Metric	Mistral	Codelama	Gemma 2	phi3
Solutions (No Errors)	36	132	128	5
Solutions (With Errors)	390	290	299	419
Total Passed Test Cases	62	178	208	10
Total Failed Test Cases	13	112	112	2
Total Error Test Cases	35	121	76	3
Pass@1 count Score = 1	20	53	61	3
Pass@1 count Score = 0	16	79	67	2
Pass@1 Score = 1/No Errors solution ratio	0.55	0.40	0.47	0.60

Since this experiment used four models for the classification problem with the use of SMOTE, the selection of the features is made using pairs of two features that give a strong correlation score and represent the most redundant information in the dataset.

RankNet is employed to rank the models' scores for the code summarization experiment. RankNet was chosen for the ranking task because it is a well-established learning-to-rank algorithm capable of modeling pairwise preferences, making it suitable for comparative evaluation of LLM-generated summaries, and providing a robust and interpretable machine learning framework for LLM evaluation.

Given the accuracy features collected for each code summarization task, the total average per-metric score is calculated for the training part of the data and considered as the base score. The testing data are evaluated on a pairwise basis, where for every possible pairing of models, the code calculates the difference in their metric scores. This difference vector then serves as the input features for the neural network.

The process moves to evaluating and ranking the models. The trained model is employed to generate a matrix of win probabilities, which shows the likelihood of each model outperforming every other model. This is achieved by feeding the difference vectors of all possible pairs into the model and averaging the predicted win probabilities. The final step is to derive a single, consolidated ranking and show the matrix of pairwise model comparison alongside the final model ranking.

The RankNet model is configured as a multi-layer perceptron classifier. This model is specifically designed to tackle the ranking problem by reframing it as a classification task. It is trained on a dataset of pairwise comparisons where the input features are the differences between two models' performance metrics on a specific task. The model then learns to predict a binary output: a 1 if the first model in the pair is better, and a 0 if the second model is better. This allows the neural network to effectively learn the subtle relationships and relative superiority of models based on their comparative scores without needing to know the absolute performance values.

3.12. Hyperparameter Settings

To optimize performance for SVM classification, this study used grid search on three main hyperparameters, which are as follows: *C*, *gamma*, and kernel. The best parameters are selected based on the accuracy score and through 5-fold cross-validation on the training set. The estimator selects the best hyperparameter values within a predefined range and is ultimately used as the final classifier.

This RankNet method systematically searches through a predefined set of hyperparameters, including the network's architecture, hidden layer sizes, the activation function (*ReLU* or *tanh*), and the L2 regularization strength (alpha). By testing all combinations of these parameters, the grid search identifies the best-performing model based on cross-validation, ensuring that the final RankNet model is well-tuned for the given task data.

3.13. Evaluation Techniques

The classification model was assessed using evaluation metrics listed in Table 7, where true positive (*TP*), false positive (*FP*), true negative (*TN*), and false negative (*FN*) notations are represented within these formulas. Additionally, Table 7 shows the AUC, MCC, and Kappa metrics that were used to evaluate results per class, along with macro-average values for AUC, precision, and recall. This study further shows the highest prediction distribution percentages among all classes.

The ranking problem is evaluated by showing a comparison matrix between all pairs of models by calculating feature difference values. A final model ranking win score is demonstrated across the selected models.

3.14. Model Training and Validation

The SVM model training process begins after the initial dataset is partitioned. An 80% training split is performed, and then a 20% testing split is used to ensure the final model is evaluated on completely unseen data. A critical first step is to convert the continuous, one-dimensional feature data from the 80% training set into a labeled, multi-class format. A 10-fold cross-validation iteration is performed on the training data. A new support vector classifier model is then instantiated and trained on 90% of the current fold's data, and the trained model is then used to predict the labels for the remaining 10% of the fold's data, which serves as the validation set.

Table 7. Classification model evaluation metrics.

Metric	Description	Formula
Accuracy	Measures the overall correctness of the model's predictions.	$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$
Precision	Measures the accuracy of positive predictions. Indicates the proportion of correctly predicted positive cases out of all predicted positives.	$\text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}$
Recall	Measures the model's ability to find all actual positive cases. Indicates the proportion of correctly predicted positive cases out of all actual positives.	$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}$
F1 score	Harmonic mean of precision and recall, providing a balanced measure of both.	$\text{F1 score} = \frac{2\cdot(\text{Precision}\cdot\text{Recall})}{\text{Precision}+\text{Recall}}$
AUC-ROC	Measures the model's ability to distinguish between classes. The area under the receiver operating characteristic curve.	Calculated from the ROC, which plots the true positive rate against the false positive rate at various threshold settings.
MCC	Metric used to evaluate the quality of classification models, especially in cases of imbalanced datasets.	$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$
Kappa	Measures the agreement between two raters (or a classifier and the true labels), accounting for the possibility of agreement occurring by chance.	Let: • P_o = observed agreement • P_e = expected agreement by chance: $P_e = \frac{(\text{TP}+\text{FP})(\text{TP}+\text{FN})+(\text{FN}+\text{TN})(\text{FP}+\text{TN})}{(\text{TP}+\text{TN}+\text{FP}+\text{FN})^2}$ Then, the Kappa coefficient (κ) is calculated as follows: $\kappa = \frac{P_o - P_e}{1 - P_e}$

The RankNet allocates 70% of the pairwise dataset for training and the remaining 30% for testing on unseen data. The average scores per metric are considered as the base score and are calculated for the training part of the data for all the models. The testing is conducted on individual summarization tasks, using unique instances with their corresponding similarity metric scores. Validation is an integral part of the training process and is handled automatically by using grid search. The methodology uses grid search with a 3-fold cross-validation for faster computational efficiency, considering the size of the dataset, making it the best choice for the validation process. During this process, the training set is partitioned into three subsets. The model is trained on two of these subsets, and its performance is validated on the remaining one. This cycle is repeated three times, ensuring that each of the three subsets serves as a validation set exactly once. This robust method of validation provides a more reliable estimate of the model's performance on new data and is crucial for selecting the final, best-performing model from all the combinations tested by the grid search.

4. Experimental Results

4.1. Machine Specification and Libraries

The code generation and code summarization experiments are conducted using a machine with the following specifications: a 2.2 GHz Intel Core i7, 16 GB 1600 MHz DDR3, and a machine with 1536 MB GPU. The Langchain library [47] version 0.3.20 is used for LLM

chain prompts and libraries for passing LLM command prompts. The TfidfVectorizer [48], from the sklearn library version 0.1.3, is used to evaluate candidate and reference strings in code description strings.

4.2. Code Generation Experimental Results

Table 6 shows the number of solutions provided by each model, including the total number of passed test cases, failed test cases, error test cases, and solutions provided with errors. Furthermore, the count of the *Pass@1* score is calculated across all the model solutions among the coding problem tasks.

4.3. Code Generation Feature Selection and Performance

Table 8 shows the average, minimum, and maximum results of the code generation features across the passed test case solutions that provided no coding solution errors from each model. The total number of features selected is 18 out of the 24 total features. As shown in Table 8, six features are dropped from the selection due to similar numeric values provided across the experiment models. Those features are *Raw_LOC*, *Raw_LLOC*, *Raw_SLOC*, *Raw_MULTI*, *Raw_BLANK*, and *Raw_SINGLE_COMMENTS*, respectively.

The selected 18 features are identified using F1 to F18 label identifiers. Similarly, the experiment models are labeled M1 to M4 to map to Mistral, CodeLlama, Gemma 2, and Phi-3 models, respectively.

Figure 6 shows comparison charts for the experiment models by selected features, which are as follows: *Maintainability_Index*, *Total_Cyclomatic_Complexity*, *CodeBLEU*, *bug*, *time*, *difficulty*, *effort*, and *Raw_LOC*. These values are calculated by identifying the raw scores and determining the normalization range across all models. Then, a normalized score between 0 and 1 is calculated, averaged, and scaled to represent a 100% score.

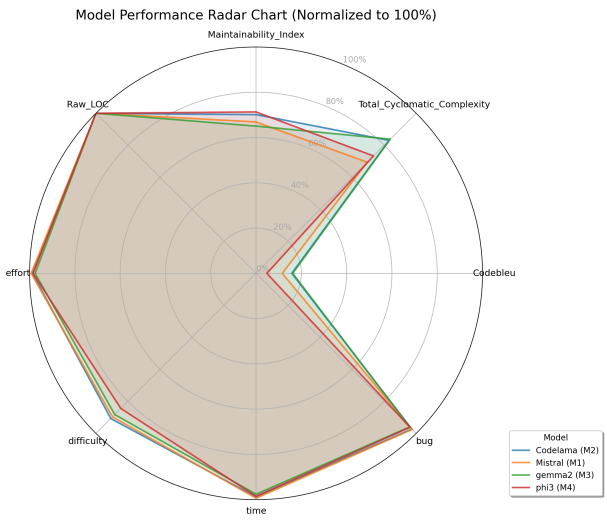

Figure 6. Selected features to evaluate experiment-model performance.

Figure 7 shows the TensorFlow-native perplexity evaluator model values for each of the four experiment models. The figure plots the perplexity score for each task instance identifier as the model solves the coding tasks sequentially in the dataset. As shown in Figure 7, DistilGPT2 gives the highest perplexity scores, and CodeGPT-small-py gives the lowest scores across models M1 to M4.

Table 8. Result of Code Generation Features provided by each model.

Feature/Models	M1 Avg	M1 Min	M1 Max	M2 Avg	M2 Min	M2 Max	M3 Avg	M3 Min	M3 Max	M4 Avg	M4 Min	M4 Max
H1 (F1)	1.0278	0.0000	3.0000	0.8636	0.0000	4.0000	1.0625	0.0000	6.0000	1.6000	1.0000	4.0000
H2 (F2)	2.0278	0.0000	8.0000	1.7879	0.0000	18.0000	2.2969	0.0000	26.0000	3.0000	2.0000	7.0000
N1 (F3)	1.0833	0.0000	4.0000	1.0152	0.0000	13.0000	1.3828	0.0000	20.0000	1.6000	1.0000	4.0000
N2 (F4)	2.0833	0.0000	8.0000	2.0000	0.0000	26.0000	2.6484	0.0000	40.0000	3.0000	2.0000	7.0000
Vocabulary (F5)	3.0556	0.0000	11.0000	2.6515	0.0000	22.0000	3.3594	0.0000	31.0000	4.6000	3.0000	11.0000
Length (F6)	3.1667	0.0000	12.0000	3.0152	0.0000	39.0000	4.0312	0.0000	60.0000	4.6000	3.0000	11.0000
Calculated_length (F7)	4.2505	0.0000	28.7549	4.3644	0.0000	83.0587	6.4148	0.0000	133.8211	7.1303	2.0000	27.6515
Volume (F8)	7.3457	0.0000	41.5132	8.1117	0.0000	173.9178	12.0023	0.0000	297.2518	11.4147	4.7549	38.0537
Difficulty (F9)	0.5324	0.0000	1.5000	0.4773	0.0000	2.8889	0.6043	0.0000	5.1429	0.8000	0.5000	2.0000
Effort (F10)	7.4093	0.0000	62.2698	14.1022	0.0000	502.4293	27.6938	0.0000	1143.2761	17.1235	2.3774	76.1075
Time (F11)	0.4116	0.0000	3.4594	0.7835	0.0000	27.9127	1.5385	0.0000	63.5153	0.9513	0.1321	4.2282
Bugs (F12)	0.0024	0.0000	0.0138	0.0027	0.0000	0.0580	0.0040	0.0000	0.0991	0.0038	0.0016	0.0127
Maintainability_Index (F13)	92.0440	81.6993	100.0000	92.8240	77.6119	100.0000	91.6062	75.9820	100.0000	93.1076	88.4230	100.0000
Raw_LOC	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Raw_LLOC	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000	2.0000
Raw_SLOC	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Raw_COMMENTS (F14)	0.1389	0.0000	1.0000	0.1136	0.0000	1.0000	0.0000	0.0000	0.0000	0.4000	0.0000	1.0000
Raw_MULTI	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Raw_BLANK	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Raw_SINGLE_COMMENTS	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Total_Cyclomatic_Complexity (F15)	1.9167	1.0000	4.0000	1.5076	1.0000	4.0000	1.4844	1.0000	4.0000	1.8000	1.0000	3.0000
Perplexity (F16)	44.4478	3.3754	779.0668	35.3702	3.0924	310.6410	54.7860	3.3754	897.5033	69.0857	8.3780	356.6891
CodeBLEU (F17)	0.0942	0.0000	0.4555	0.1285	0.0000	0.8137	0.1318	0.0000	0.6888	0.0390	0.0000	0.1613
pass@1 (F18)	0.5556	0.0000	1.0000	0.4015	0.0000	1.0000	0.4766	0.0000	1.0000	0.6000	0.0000	1.0000

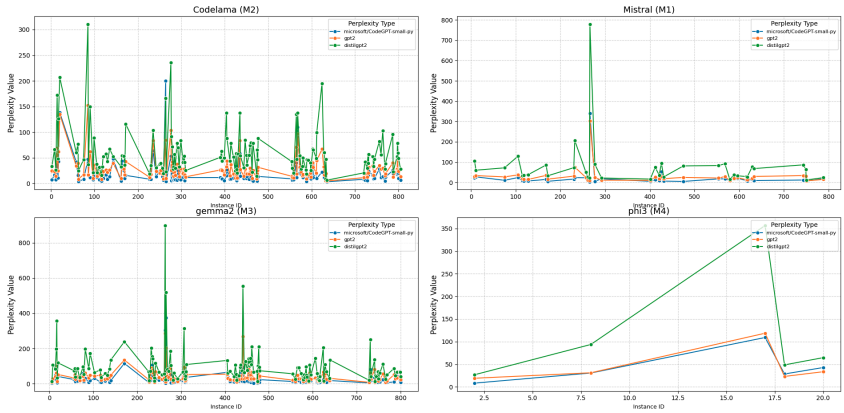

Figure 7. Perplexity evaluator-model values for each experiment model.

4.4. Code Generation Classification Results

Figure 8 illustrates the correlation matrix scores for all of the selected features. The matrix shows a strong correlation among the values from F1 to F12, as demonstrated by the numeric coefficient values. F14 to F18 show the lowest correlation score among all the features.

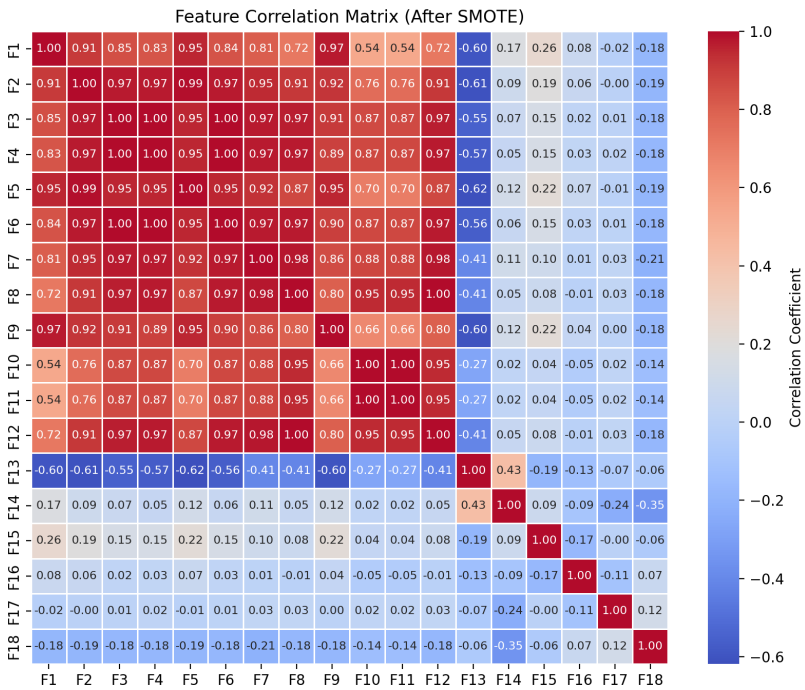

Figure 8. Feature correlation matrix.

Table 9 shows the top 10 selected pairs of features from the classification results with strong corresponding correlation scores after applying SMOTE. This study refers to classes 0, 1, 2, and 3 to represent SVM classification results for models M1 to M4.

The total number of actual resampled class distribution samples was 132 for each class. After applying SMOTE, the highest predicted sample percentage reached 69.7%, for a total of 368 samples, for the feature pair F10 and F11.

The maximum accuracy reached was for features F2 and F5 with 49%. The lowest accuracy reached was 37% for the feature pair F10 and F11. Kappa and MCC scores were highest with the feature pair F2 and F5, reaching 32% and 34%, respectively.

F3 and F4 showed the highest Macro-Avg F1 score and Macro-Avg AUC-ROC with percentages of 44% and 72%, respectively.

The macro-average score for precision ranged from 50.7% to 57.5%. The macro-average score for recall was lower than the precision percentage, ranging from 38% to 49%. The highest precision score reached 90% for the F10 and F11 feature pair for class 2. The recall score reached up to 92% for both feature pairs (F7, F12) and (F7, F8), both for class 3. Furthermore, the F1 score reached 0.64 for class 3 for the same feature pairs.

Figure 9 presents the ROC curves for the top four feature pairs with strongly correlated scores, which are (F10, F11), (F8, F12), (F4, F6), and (F3, F6). All of these pairs reached the highest AUC for class 3, ranging from 0.72 to 0.86. Furthermore, the same figure shows that class 2 reached the lowest score among all other classes, ranging from 0.60 to 0.66.

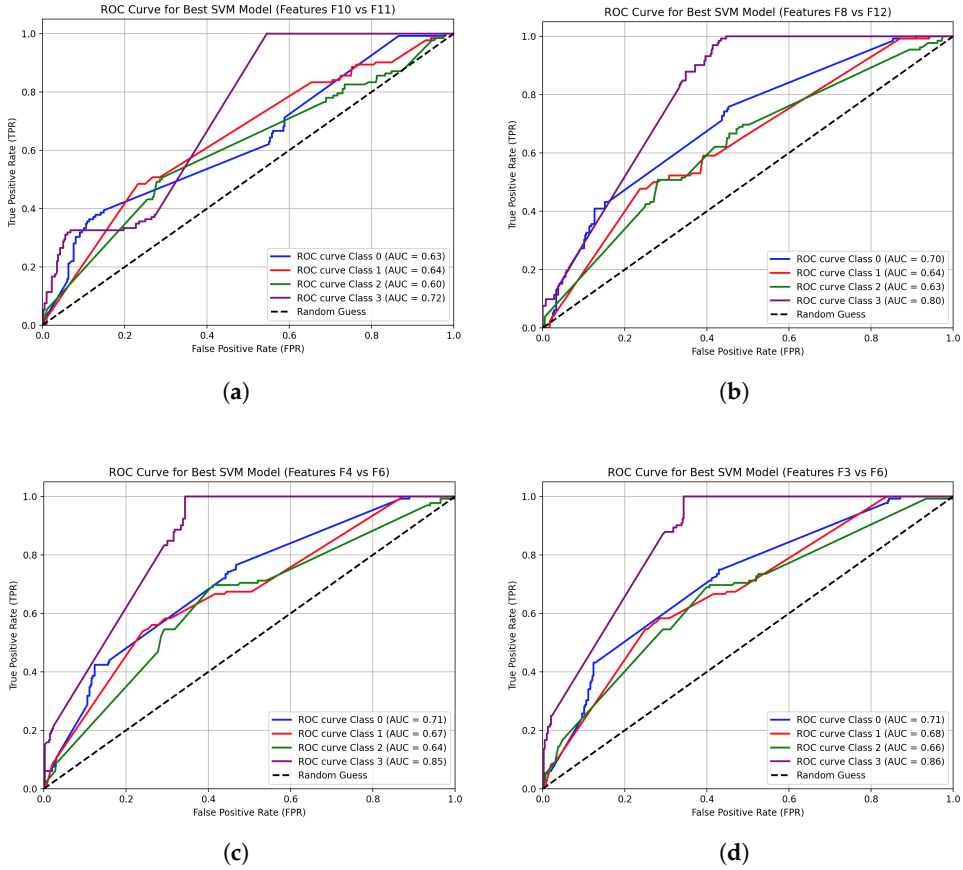

Figure 9. ROC curves for the top four features in the SVM model: (**a**) ROC Curve for Feature pair (F10,F11); (**b**) ROC Curve for Feature pair (F8,F12); (**c**) ROC Curve for Feature pair (F4,F6); (**d**) ROC Curve for Feature pair (F3,F6).

In evaluating the efficacy of the synthetic variant data samples generated via SMOTE, we conducted a comparative analysis against a non-SMOTE approach using a weighted SVM implementation that automatically calculates weights inversely proportional to class frequencies. The initial experiments revealed a significant bias toward class 0 compared with the other classes, with a maximum accuracy of 12.09%. Conversely, the SMOTE-enhanced experiments demonstrated that by artificially balancing the class distribution, the classifier was better able to learn the decision boundaries of underrepresented instances. These results confirm that the inherent class imbalance in the raw data severely suppresses the predictive power of standard algorithms, necessitating the use of over-sampling to ensure robust performance across all selected models. Table 10 shows the results for the weighted SVM code generation classification for the top 10 feature pairs with the highest correlation scores.

Table 9. Results of code generation classification for the top 10 feature-pair correlation scores.

Features Pairs	Correlation Score	Accuracy	Kappa	MCC	Macro-Avg F1 Score	Macro-Avg AUC-ROC	Macro-Avg Precision	Macro-Avg Recall	Highest Predicted Class Distribution
F10 and F11	1.0	0.3788	0.1717	0.2171	0.3370	0.6509	0.5875	0.3800	69.70%
F8 and F12	1.0	0.4356	0.2475	0.2693	0.3838	0.6926	0.5175	0.4375	48.11%
F4 and F6	0.9994	0.4792	0.3056	0.3258	0.4361	0.7143	0.5250	0.4800	45.27%
F3 and F6	0.9979	0.4792	0.3056	0.3267	0.4353	0.7272	0.5300	0.4800	45.83%
F3 and F4	0.9951	0.4848	0.3131	0.3342	0.4417	0.7292	0.5375	0.4875	45.45%
F2 and F5	0.9915	0.4905	0.3207	0.3419	0.4588	0.7050	0.5525	0.4900	46.97%
F7 and F12	0.9778	0.4792	0.3056	0.3320	0.4246	0.7270	0.5750	0.4775	47.35%
F7 and F8	0.9778	0.4792	0.3056	0.3320	0.4246	0.7270	0.5750	0.4775	47.35%
F2 and F6	0.9746	0.4867	0.3157	0.3363	0.4497	0.7086	0.5525	0.4875	45.45%
F2 and F4	0.9744	0.4640	0.2854	0.3066	0.4125	0.7040	0.5075	0.4650	46.78%

Table 10. Results of code generation classification for the top 10 feature-pair correlation scores using a non-SMOTE weighted SVM.

Features Pairs	Correlation Score	Accuracy	Kappa	MCC	Macro-Avg F1 Score	Macro-Avg AUC-ROC	Macro-Avg Precision	Macro-Avg Recall	Highest Predicted Class Distribution
F8 and F12	1.0	0.1209	0.0014	0.0105	0.0545	0.5024	0.0306	0.2500	98.90%
F10 and F11	1.0	0.1209	0.0000	0.0000	0.0539	0.5197	0.0302	0.2500	100.00%
F4 and F6	0.9996	0.1209	0.0027	0.0149	0.0550	0.4413	0.0309	0.2500	97.80%
F3 and F6	0.9986	0.1209	0.0027	0.0149	0.0550	0.4593	0.0309	0.2500	97.80%
F3 and F4	0.9968	0.1209	0.0027	0.0149	0.0550	0.4605	0.0309	0.2500	97.80%
F2 and F5	0.9917	0.0769	−0.0178	−0.0338	0.0443	0.5030	0.0257	0.1591	74.73%
F7 and F8	0.9862	0.1209	0.0027	0.0149	0.0550	0.4854	0.0309	0.2500	97.80%
F7 and F12	0.9862	0.1209	0.0027	0.0149	0.0550	0.4854	0.0309	0.2500	97.80%
F4 and F7	0.9808	0.1209	0.0027	0.0149	0.0550	0.4641	0.0309	0.2500	97.80%
F6 and F7	0.9799	0.1209	0.0027	0.0149	0.0550	0.4615	0.0309	0.2500	97.80%

4.5. Code Summarization Experimental Results

Table 11 shows the results of ROUGE and BERT scores for the code summarization experiment across a total of 301 data points. The M3 model reached the highest BERT score of 0.87, while M4 had the lowest score of 0.63. For the ROUGE metrics, the M2 model reached its highest score of 0.66 for the ROUGE-1 metric, M3 reached 0.43 for the ROUGE-2 metric, and M2 reached 0.57 for the ROUGE-L metric. Similar to the BERT score result, M4 also showed the lowest score among all of the ROUGE evaluation metric scores.

Table 11. Results of code summarization evaluation metric features.

Model	BERT F1 Avg	BERT F1 Min	BERT F1 Max	ROUGE-1 F1 Avg	ROUGE-1 F1 Min	ROUGE-1 F1 Max	ROUGE-2 F1 Avg	ROUGE-2 F1 Min	ROUGE-2 F1 Max	ROUGE-L F1 Avg	ROUGE-L F1 Min	ROUGE-L F1 Max
Mistral (M1)	0.7922	0.7032	0.8478	0.3337	0.1875	0.5294	0.1169	0	0.3137	0.2551	0.1250	0.4705
Codelama (M2)	0.7865	0.6775	0.8572	0.2689	0.0493	**0.6666**	0.1026	0	0.3437	0.2217	0.0493	**0.5757**
Gemma 2 (M3)	0.8098	0.7211	**0.8784**	0.3580	0.1649	0.5714	0.1386	0	**0.4324**	0.2848	0.0824	0.5128
phi3 (M4)	0.6737	0.6360	0.7023	0.0952	0.0263	0.1481	0	0	0	0.0746	0.0263	0.1111

Note: Bold text highlights the maximum F1 value across the models.

Figure 10 shows all of the BERT and ROUGE scores per task identifier for all the models as they were processed sequentially. The same figure shows the four models' BERT and ROUGE values on a task-level basis.

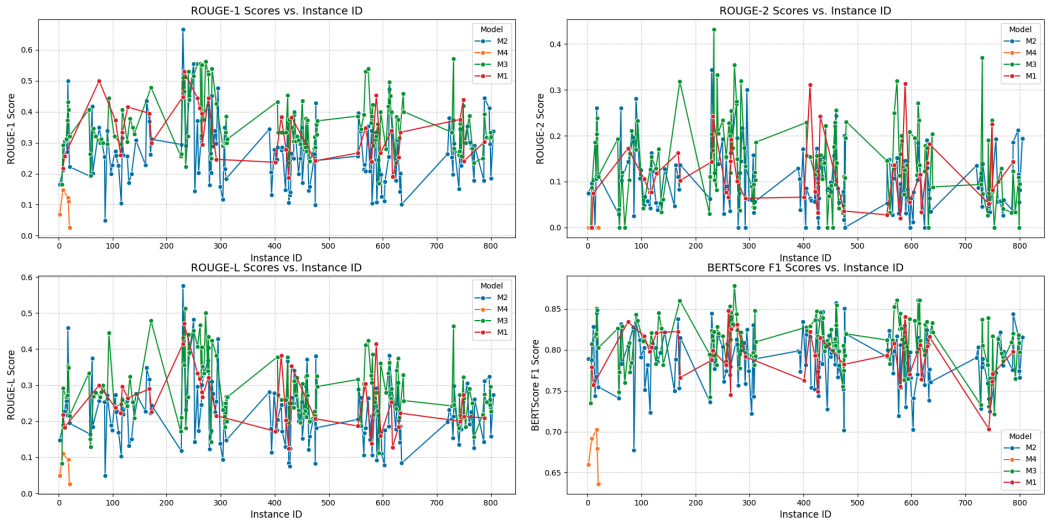

Figure 10. BERT and ROUGE data point score visualizations per task identifier.

4.6. Code Summarization Ranking Results

In total, 202 unique instances were extracted, with 141 data point samples used to calculate the base average scores across the models for training. The other 61 data point samples were treated as unseen data for the final ranking comparison.

After calculating the average base score, the data were then combined for 62 unique instances from the 4 models, generating pairwise training data in which 39 pairwise samples were created with 4 features each. Finally, the RankNet approach was run with GridSearchCV, fitting 3 folds for each of 12 candidates, for a total of 36 fits.

Table 12 shows the models' pairwise RankNet comparison matrix. By calculating the sum of aggregated win scores for each model, the final model ranking is M3 with 1.93, M4 with 1.66, M2 with 1.2, and M1 with 0.65. Figure 11 shows the final RankNet sum of win probabilities for each model, which is calculated from each row in Table 12.

Table 12. RankNet comparison matrix.

	M1	**M2**	**M3**	**M4**
M1	X	0.4634	0.1892	0
M2	0.3415	X	0.2037	0.6645
M3	0.7261	0.5436	X	0.6666
M4	1.0	0.3335	0.3333	X

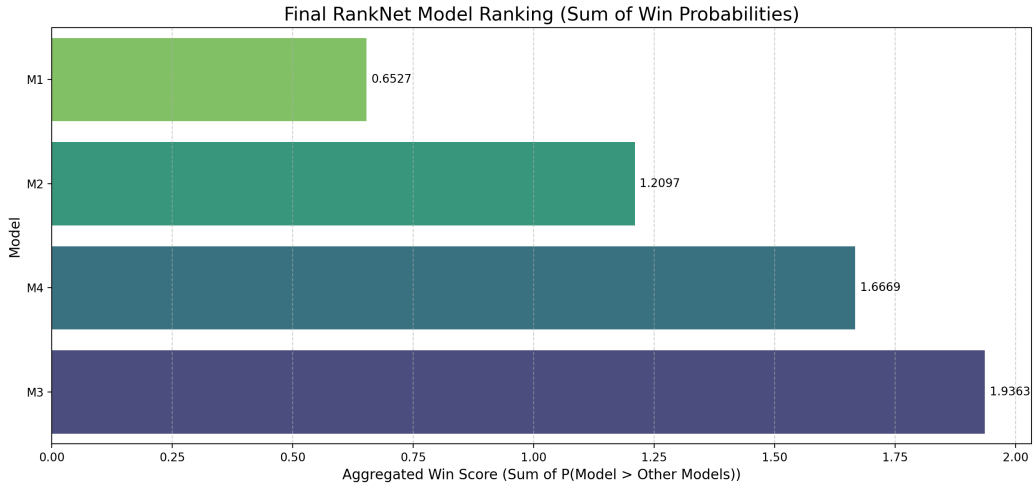

Figure 11. Sum of win probabilities for the final RankNet model ranking.

5. Discussions

The results obtained from the code generation SVM classification experiment provide a comprehensive overview of predicting LLM classes across pairs of highly correlated features. The classification accuracy reached up to 49.05% among the top 10 pairs of correlated features, with a high AUC score of 86% among the top 4 pairs of correlated features. The high precision (90%) and recall (92%) indicate few false positives or negatives in identifying the predicted model class, and the distribution percentage reaches up to 69.70%.

As some classes have more samples than others, it is important to measure an overall score that reflects the model's performance across all classes equally. The macro-averaged AUC-ROC results, ranging from 65% to 72%, are a strong indicator of the model's ability to provide fair classification predictions across all models. Furthermore, another indicator for an imbalanced dataset is the macro-averaged F1 score, as it reflects general performance and not just performance on the majority classes, reaching a maximum of 45.88%. The average accuracy scores could be explained by the model not learning enough about the minority classes due to insufficient training data for those classes. Even with SMOTE generating synthetic samples, some of the LLMs provided few coding solutions from the dataset, which increases the likelihood of incorrect predictions for minority classes, lowering overall accuracy. It is also noted that minority classes often have low recall and low precision, which eventually affects the overall F1 scores for those classes.

To validate the effectiveness of the proposed SVM framework, a baseline comparison was conducted using a simple selection strategy based solely on pass@1 accuracy. From Table 6, the pass@1 score ratio of error-free solutions indicates selection of Phi-3 with 60%, Mistral with 55%, Gemma 2 with 47%, and CodeLlama with 40%. In contrast, based on the SVM experiment with the highest correlated feature pair, the model selection prediction class order is 69.70% for CodeLlama, 18.94% for Mistral, 9.47% for Phi-3, and 1.89% for Gemma 2.

The experimental results indicate that the proposed approach's prediction percentage variance across the models outperforms the pass@1 baseline in terms of class prediction identification percentage. This confirms that machine learning-based feature analysis provides more reliable LLM selection than heuristic or single-metric approaches.

The results obtained from the code summarization RankNet ranking show close results for models M3 and M4, with win probabilities of 1.93 and 1.66, respectively. The RankNet comparison matrix is highest for the (M3, M1) pair with a score of 0.72. The (M2, M4) and (M3, M4) pairs had similar scores of 0.6645 and 0.66, respectively. In the results, the (M4, M1) pair reached a score of 1, indicating M4's ability to outperform M1.

Practical Application and Experiment Reproducibility

In practical settings, the proposed framework can be used to select LLMs based on application-specific priorities. For example, developers prioritizing code correctness and maintainability may prefer models with higher classification confidence scores, while tasks emphasizing documentation quality may benefit from models with higher summarization rankings. This illustrates that different models may be preferred depending on the task requirements and the code metrics prioritized by stakeholders. Other applications include integrating the selection of best-suited LLMs into Integrated Development Environments (IDEs), which can enable adaptive learning, real-time code suggestions, and automatic documentation generation based on software metric features or developers' preferences in code generation and summarization domains. Future studies could explore how the proposed selection framework interacts with IDE plugins to optimize workflow efficiency and developer productivity.

LLM selection can impact real-life software engineering outcomes, such as code quality, maintainability, and adherence to emerging coding standards. By integrating LLMs with higher performance scores, development teams can reduce defect rates, improve code readability, and establish benchmarks that inform future coding standards and automated development practices.

All experiments were conducted using publicly available datasets and open-source LLM implementations. Model versions, hyperparameters, evaluation metrics, and random seed configurations are explicitly documented to support reproducibility, with the code base shared through GitHub [37]. The MBPP dataset and evaluation pipeline can be reused to replicate the reported results, enabling transparent and reproducible research.

6. Conclusions

The findings of this research indicate that the selection of the best-suited LLM for code generation and code summarization could utilize machine learning classification and ranking algorithms as selection criteria. The selection criteria are dependent on a set of software metric features and accuracy scores from summarization similarities. The maximum accuracy reached is 49% for the code-generation metrics experiment. The highest precision score reached 90%, and the recall score reached up to 92%. The highest AUC score was reached with an impressive percentage of 86% among the top four pair correlated features. In the code summarization experiment, the M3 model got the highest ranking with a 1.93 score. The second-highest model was M4 with a 1.66 score. This classification and ranking methodology could be used by other researchers to provide different techniques and insights on LLM selection for different code-generation and summarization metrics.

Future research will focus on incorporating larger and more diverse datasets, including multilingual and multimodal code benchmarks. Alternative machine learning models, such as ensemble classifiers and neural ranking approaches, will be explored to improve accuracy

and robustness. Additionally, execution-based evaluation metrics and human-in-the-loop assessments will be integrated to capture functional correctness and real-world usability.

While the current study focuses on model-centric evaluation, incorporating user feedback and interactive performance metrics could provide additional insights into LLM usability, real-time coding support, and effectiveness in collaborative software development environments.

In real-world applications, long-context approaches are critical; unlike isolated snippets, professional software development involves multi-file dependencies and extensive libraries that often exceed standard LLM token limits. Handling this requires sophisticated strategies—such as Sliding Window Attention [49], RAG-based (retrieval-augmented generation) context filtering [50], or linear attention mechanisms [51]—to ensure the LLM maintains project-level awareness during code generation and summarization. While the current experimental setup focused on modular tasks, where these techniques were not strictly required, evaluating the robustness of these long-context strategies for complex, large-scale repository generation remains a vital direction for future research.

Limitations. This study is not without limitations; it must acknowledge the following:

- LLMs selected in the experiment differ in coding and summarization capabilities. This might affect the overall score results and could give misleading accuracy values. In addition, other prompt types (e.g., few-shot, chain-of-thought, etc.) and the choice of wording could influence the results.
- Features selected for coding prediction results are those that show a highly correlated score. The features selected might not be the best for measuring coding capabilities and may not be the best to use to judge models in this domain.
- Synthetic samples may not represent the true distribution, as SMOTE only addresses quantity imbalance and not sample quality, which could lead to overfitting or poor generalization.
- The generalizability of the study findings to other datasets could give better results.
- The final scores of code summarization results are built from BERT and ROUGE baseline scores, which differ across models and could affect the overall score. Similarly, if evaluators are fundamentally different (e.g., one measures accuracy, another measures perplexity), averaging could distort the meaning.
- The selection of machine learning methodology, hyperparameters, and cross-validation percentage could affect the accuracy scores and could eventually lead to better classification and ranking results.
- The proposed machine learning-based framework is largely programming-language agnostic, as it relies on feature extraction, classification, and ranking rather than language-specific heuristics. However, metric distributions, summarization behavior, and the implementation of different software libraries may vary across programming languages and could affect the experiment results.
- The observed maximum classification accuracy of 49% for the code generation experiment suggests that code complexity and quality metrics alone provide limited discriminative power when distinguishing between multiple LLMs with overlapping capabilities. This indicates that while such metrics capture structural and maintainability aspects, they may not fully reflect semantic correctness or problem-solving strategies employed by different models. Incorporating additional features—such as execution-based correctness measures, token-level confidence scores, or embedding-based semantic representations—may improve classification performance.

Author Contributions: Conceptualization, methodology, experiment, data collection, data analysis, visualization, software coding, writing—original draft preparation, and editing, H.M.; model building, data modeling, writing—original draft, review, editing, and supervision, M.H.; formal analysis, validation, funding acquisition, and supervision, B.A.Y.A. and A.I.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The code will be finalized and made publicly available online upon acceptance of the paper.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Xu, J.; Zhang, H.; Yang, Y.; Yang, L.; Cheng, Z.; Lyu, J.; Liu, B.; Zhou, X.; Bacchelli, A.; Chiam, Y.K.; et al. One Size Does Not Fit All: Investigating Efficacy of Perplexity in Detecting LLM-Generated Code. *ACM Trans. Softw. Eng. Methodol.* **2024**. [CrossRef]
2. Jiang, J.; Wang, F.; Shen, J.; Kim, S.; Kim, S. A survey on large language models for code generation. *arXiv* **2024**, arXiv:2406.00515. [CrossRef]
3. Shao, Y.; Huang, Y.; Shen, J.; Ma, L.; Su, T.; Wan, C. Are LLMs Correctly Integrated into Software Systems? *arXiv* **2024**, arXiv:2407.05138.
4. Haque, M.A. Llms: A game-changer for software engineers? *Benchcouncil Trans. Benchmarks Stand. Eval.* **2025**, *5* , 100204.
5. Das, J.K.; Mondal, S.; Roy, C.K. Why Do Developers Engage with ChatGPT in Issue-Tracker? Investigating Usage and Reliance on ChatGPT-Generated Code. In *Proceedings of the 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*; IEEE: Montreal, QC, Canada, 2025; pp. 68–79.
6. Tian, H.; Lu, W.; Li, T.O.; Tang, X.; Cheung, S.C.; Klein, J.; Bissyandé, T.F. Is ChatGPT the ultimate programming assistant–how far is it? *arXiv* **2023**, arXiv:2304.11938.
7. Liu, J.; Xia, C.S.; Wang, Y.; Zhang, L. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 21558–21572.
8. Liu, Y.; Le-Cong, T.; Widyasari, R.; Tantithamthavorn, C.; Li, L.; Le, X.B.D.; Lo, D. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–26. [CrossRef]
9. Sepidband, M.; Taherkhani, H.; Wang, S.; Hemmati, H. Enhancing LLM-Based Code Generation with Complexity Metrics: A Feedback-Driven Approach. *arXiv* **2025**, arXiv:2505.23953.
10. Khan, M.F.A.; Ramsdell, M.; Falor, E.; Karimi, H. Assessing the promise and pitfalls of chatgpt for automated code generation. *arXiv* **2023**, arXiv:2311.02640. [CrossRef]
11. Hu, W.; Duan, J.; Wei, C.; Zhang, L.; Zhang, Y.; Xu, K. Dynacode: A dynamic complexity-aware code benchmark for evaluating large language models in code generation. *arXiv* **2025**, arXiv:2503.10452.
12. Biswas, S.; Singh, G.; Maiti, B.; Ezugwu, A.E.S.; Saleem, K.; Smerat, A.; Abualigah, L.; Bera, U.K. Integrating Differential Evolution into Gazelle Optimization for advanced global optimization and engineering applications. *Comput. Methods Appl. Mech. Eng.* **2025**, *434*, 117588. [CrossRef]
13. Mahmud, T.; Duan, B.; Pasareanu, C.; Yang, G. Enhancing llm code generation with ensembles: A similarity-based selection approach. *arXiv* **2025**, arXiv:2503.15838. [CrossRef]
14. Huynh, N.; Lin, B. Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications. *arXiv* **2025**, arXiv:2503.01245.
15. Mostafa, A.; Nahid, R.A.; Mulder, S. How Different Tokenization Algorithms Impact LLMs and Transformer Models for Binary Code Analysis. *arXiv* **2025**, arXiv:2511.03825. [CrossRef]
16. Xu, Z.; Sheng, V.S. Detecting AI-generated code assignments using perplexity of large language models. In Proceedings of the AAAI Conference on Artificial Intelligence, Vancouver, BC, Canada, 20–27 February 2024; Volume 38, pp. 23155–23162.
17. Wang, Q.; Parnin, C.; Orso, A. Evaluating the usefulness of ir-based fault localization techniques. In Proceedings of the 2015 International Symposium on Software Testing and Analysis, Baltimore, MD, USA, 13–17 July 2015; pp. 1–11.
18. Bo, L.; Ji, W.; Sun, X.; Zhang, T.; Wu, X.; Wei, Y. Chatbr: Automated assessment and improvement of bug report quality using chatgpt. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; pp. 1472–1483.

19. Acharya, J.; Ginde, G. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. *arXiv* **2025**, arXiv:2504.18804. [CrossRef]
20. Sureka, A.; Indukuri, K.V. Linguistic analysis of bug report titles with respect to the dimension of bug importance. In Proceedings of the Third Annual ACM Bangalore Conference, Bangalore, India, 22–23 January 2010; pp. 1–6.
21. Kang, S.; Milliken, L.; Yoo, S. Identifying inaccurate descriptions in llm-generated code comments via test execution. *arXiv* **2024**, arXiv:2406.14836. [CrossRef]
22. Kumar, A.; Haiduc, S.; Das, P.P.; Chakrabarti, P.P. LLMs as Evaluators: A Novel Approach to Evaluate Bug Report Summarization. *arXiv* **2024**, arXiv:2409.00630. [CrossRef]
23. Colavito, G. Foundation Models for Automatic Issue Labeling. In *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*; IEEE: Ottawa, ON, Canada, 2025, pp. 127–131.
24. Shivashankar, K.; Haugerud, K.M.; Martini, A. Enhancing Task Prioritization in Software Development Issues Tracking system. *J. Softw. Evol. Process* **2025**, *37*, e70068. [CrossRef]
25. Chen, Z.; Nava-Camal, V.; Suleiman, A.; Tang, Y.; Hou, D.; Shang, W. An Empirical Study on the Capability of LLMs in Decomposing Bug Reports. *arXiv* **2025**, arXiv:2504.20911. [CrossRef]
26. Ciepielowski, L. An LLM-Based Tool for Knowledge Retrieval from (Heterogeneous) Issue Tracking Systems. Ph.D. Thesis, Universit̊at Hamburg, Hamburg, Germany, 2024.
27. Plein, L.; Bissyandé, T.F. Can LLMs demystify bug reports? *arXiv* **2023**, arXiv:2310.06310. [CrossRef]
28. Sun, W.; Miao, Y.; Li, Y.; Zhang, H.; Fang, C.; Liu, Y.; Deng, G.; Liu, Y.; Chen, Z. Source code summarization in the era of large language models. *arXiv* **2024**, arXiv:2407.07959. [CrossRef]
29. Wu, Y.; Wan, Y.; Chu, Z.; Zhao, W.; Liu, Y.; Zhang, H.; Shi, X.; Jin, H.; Yu, P.S. Can large language models serve as evaluators for code summarization? *IEEE Trans. Softw. Eng.* **2025**, *51*, 3205–3217. [CrossRef]
30. Apollinaire, I.O.A. How to Choose the Best AI LLM: A Guide to Navigating the Diversity of Models. *J. Inf. Syst. Eng. Manag.* **2024**, *10*, 221–232.
31. Cheng, J.; Liu, F.; Wu, C.; Zhang, L. Adaptivellm: A framework for selecting optimal cost-efficient llm for code-generation based on cot length. In Proceedings of the 16th International Conference on Internetware, Trondheim, Norway, 20–22 June 2025; pp. 461–473.
32. Arora, C.; Sayeed, A.I.; Licorish, S.; Wang, F.; Treude, C. Optimizing LLMs for Code Generation: Which Hyperparameter Settings Yield the Best Results? In *Proceedings of the 2024 31st Asia-Pacific Software Engineering Conference (APSEC)*; IEEE: Chongqing, China, 2024; pp. 281–290.
33. Fan, Z.; Ruan, H.; Mechtaev, S.; Roychoudhury, A. Oracle-guided Program Selection from Large Language Models. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Vienna, Austria, 16–20 September 2024; pp. 628–640.
34. Su, J.; Deng, L.; Wen, C.; Qin, S.; Tian, C. Cfstra: Enhancing configurable program analysis through llm-driven strategy selection based on code features. In *Proceedings of the International Symposium on Theoretical Aspects of Software Engineering*; Springer: Guiyang, China, 2024; pp. 374–391.
35. Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. Program synthesis with large language models. *arXiv* **2021**, arXiv:2108.07732. [CrossRef]
36. Polozov, A. *Sanitized-Mbpp*. 18 August 2021. Available online: https://github.com/google-research/google-research/blob/master/mbpp/sanitized-mbpp.json (accessed on 23 November 2025).
37. Mahfoodh, H. *Hussain Mahfodh Github Page*. 19 August 2024. Available online: https://github.com/hmahfoodh/AI_ML_Data_Engineering/tree/main/LLM (accessed on 23 November 2025).
38. Ollama Team. Mistral Model Library Entry. Available online: https://ollama.com/library/mistral (accessed on 28 January 2026).
39. Ollama Team. Gemma 2 Model Library Entry. 2024. Available online: https://ollama.com/library/gemma2 (accessed on 28 January 2026).
40. Ollama Team. Phi-3 Model Library Entry. 2024. Available online: https://ollama.com/library/phi3 (accessed on 28 January 2026).
41. Ollama Team. CodeLlama Model Library Entry. 2023. Available online: https://ollama.com/library/codellama (accessed on 28 January 2026).
42. Khan, A.A.; Mahmood, A.; Amralla, S.M.; Mirza, T.H. Comparison of software complexity metrics. *Int. J. Comput. Netw. Technol.* **2016**, *4, 19-26* . [CrossRef]
43. Xie, Y.; He, M.; Ma, T.; Tian, W. Optimal distributed parallel algorithms for deep learning framework tensorflow. *Appl. Intell.* **2022**, *52*, 3880–3900. [CrossRef]
44. Microsoft. CodeGPT-small-py on Hugging Face Model Hub. 2025. Available online: https://huggingface.co/microsoft/CodeGPT-small-py (accessed on 8 November 2025).

45. Saidi, I.; Alshammari, A.; Alosaimi, A.; Alqahtani, F. The Impact of Design on the Performance of Large Language Models. *Inf. Softw. Technol.* **2020**, *123*, 106297. [CrossRef]
46. Hugging Face. distilgpt2 on Hugging Face Model Hub. 2025. Available online: https://huggingface.co/distilbert/distilgpt2 (accessed on 8 November 2025).
47. Mavroudis, V. LangChain. *Preprints* **2024**. [CrossRef]
48. Zhao, G.; Liu, Y.; Zhang, W.; Wang, Y. TFIDF based feature words extraction and topic modeling for short text. In Proceedings of the 2018 2nd International Conference on Management Engineering, Software Engineering and Service Sciences, Wuhan, China, 13–15 January 2018; pp. 188–191.
49. Fu, Z.; Song, W.; Wang, Y.; Wu, X.; Zheng, Y.; Zhang, Y.; Xu, D.; Wei, X.; Xu, T.; Zhao, X. Sliding Window Attention Training for Efficient Large Language Models. *arXiv* **2025**, arXiv:2502.18845. [CrossRef]
50. Wang, Z.; Liang, Z.; Shao, Z.; Ma, Y.; Dai, H.; Chen, B.; Mao, L.; Lei, C.; Ding, Y.; Li, H. InfoGain-RAG: Boosting Retrieval-Augmented Generation via Document Information Gain-based Reranking and Filtering. *arXiv* **2025**, arXiv:2509.12765.
51. Han, I.; Jayaram, R.; Karbasi, A.; Mirrokni, V.; Woodruff, D.P.; Zandieh, A. Hyperattention: Long-context attention in near-linear time. *arXiv* **2023**, arXiv:2310.05869.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.